

Return by Death — Kịch bản thuyết trình

Haiyahhh

Đây là kịch bản thuyết trình cho `slides/return-by-death.tex`. Slide vẫn để tiếng Anh, còn kịch bản này bằng tiếng Việt để trình bày. Mỗi mục dưới đây tương ứng với một slide, theo đúng thứ tự, và ghi lại nội dung cần nói — bản thân slide chỉ có các gạch đầu dòng ngắn gọn. Số slide ở đây **nhều hơn** bản deck hiện tại vì phần kiến thức nền đã được tách nhỏ ra cho dễ theo dõi hơn — deck sẽ được cập nhật theo kịch bản này sau. Dòng [Gợi ý: ...] đánh dấu những slide đang dùng ảnh placeholder, cần thay bằng ảnh thật trước khi thuyết trình.

1 Tiêu đề & Chương trình

Slide 1 — Title

Slide tiêu đề, ảnh thumbnail, thẻ tác giả.

Chào mọi người. Mình là Haiyahhh, hôm nay mình sẽ bảo vệ thử thách CTF của mình, tên thử thách là "Return By Death", hưởng ứng cho dịp Re:Zero anime mùa 4 vừa ra mắt và có số điểm 10/10 trên IMBD.

[Gợi ý: thay `rbd-thumbnail.png` bằng `banner/logo thật của thử thách`]

Slide 2 — Agenda

Mục lục.

Mình sẽ trình bày theo trình tự sau: ý tưởng của thử thách, kiến thức nền cần có để theo dõi phần sau, kiến trúc hệ thống, toàn bộ kill chain, lý do mình thiết kế như vậy, và cuối cùng là demo trực tiếp.

2 Khái niệm

Slide 3 — Section divider — Concept

Slide chuyển mục, tự sinh.

(Không cần nói gì — chỉ là slide chuyển mục.)

Slide 4 — Theme & Core Lesson

Bối cảnh Re:Zero, 4 lỗ hổng, `white/gray-box`.

Tên thử thách lấy từ “Return by Death” trong Re:Zero: nhân vật chính Subaru chết đi rồi quay lại một checkpoint và trong challenge này, mình đã mô phỏng lại khả năng này bằng việc sử dụng kubernetes khi mà mỗi một pod trong cluster bị hủy thì kubernetes sẽ tự động tạo ra một pod mới. Bài học cốt lõi xâu chuỗi bốn thứ: DOM Clobbering, một race condition kiểu TOCTOU,

hành vi vòng đời Pod của Kubernetes, và lỗ hổng PyYAML insecure deserialization. Đây là thử thách white-box, mình cung cấp toàn bộ mã nguồn và manifest Kubernetes cho người chơi.

3 Kiến thức nền cần biết

Slide 5 — Section divider — Background You Need

Slide chuyển mục, tự sinh.

Trước khi đi vào kill chain, chúng ta sẽ cùng nhau đi qua một số những kiến thức cơ bản mà người chơi sẽ cần biết khi giải bài này.

Slide 6 — Kubernetes 101 — What Is It?

So sánh với docker-compose, ẩn dụ cụm máy kiểu HTB.

Nếu bạn nào từng dùng docker-compose thì chắc cũng dễ hình dung Kubernetes hơn. docker-compose cho mình định nghĩa nhiều container chạy cùng lúc bằng một file YAML, nhưng tất cả các container đó đều chạy trên đúng một máy. Kubernetes làm một việc tương tự — cũng định nghĩa mọi thứ bằng YAML — nhưng thay vì chỉ một máy, nó điều phối trên cả một cụm gồm nhiều máy khác nhau, gọi là node. Bạn nào từng chơi mấy lab kiểu Hack The Box có nguyên một cụm nhiều box nối mạng nội bộ với nhau, kiểu lab Active Directory có DC với vài workstation, thì K8s cluster cũng y hệt vậy — một tập hợp nhiều máy nối mạng với nhau, và Kubernetes chính là "người quản lý" quyết định container nào chạy trên máy nào.

Slide 7 — Kubernetes 101 — Architecture

Control plane, worker node, kubelet, Kind.

Về mặt kiến trúc, một cụm Kubernetes có hai loại máy. Control plane là "bộ não" của cụm, giữ trạng thái mong muốn của toàn hệ thống và quyết định việc gì cần làm. Worker node là những máy thực sự chạy container của mình — mỗi node có một agent tên là kubelet, nhận lệnh từ control plane rồi khởi động, theo dõi, và khởi động lại container khi cần. Khi một Pod được lên lịch, control plane sẽ chọn một node còn tài nguyên trống rồi giao Pod đó cho kubelet trên node đó chạy. Thử thách này dùng Kind, tức là Kubernetes chạy bên trong Docker, nên toàn bộ "cụm nhiều máy" mình vừa nói thực chất là các container Docker giả lập node, chạy gọn trên một máy duy nhất — vẫn đúng kiến trúc control plane / node, chỉ là thu nhỏ lại cho dễ deploy thôi.

Slide 8 — Kubernetes 101 — Volumes: What & Why

Container ephemeral, vì sao cần Volume.

Có một điều quan trọng cần nắm trước: bản thân container là "ephemeral", tức là mọi thứ ghi vào filesystem của container sẽ biến mất ngay khi container đó bị thay thế — kể cả khi Kubernetes chỉ đơn giản khởi động lại một container mới trong đúng Pod cũ, giống hệt cơ chế Return by Death mình vừa nói lúc nãy. Volume trong Kubernetes chính là cách giải quyết chuyện đó: một vùng lưu trữ được gắn (mount) vào Pod, tồn tại độc lập với vòng đời của từng container bên trong Pod. Có nhiều loại Volume tùy vào bạn muốn dữ liệu sống bao lâu và ở đâu — loại mà thử thách của mình dùng là emptyDir.

Slide 9 — Kubernetes 101 — emptyDir Specifically

Vòng đời gắn với Pod chứ không phải container – chính là cơ chế mình khai thác.

`emptyDir` là loại Volume đơn giản nhất: nó được tạo rỗng ngay khi Pod được lên lịch, và sống chừng nào Pod đó còn sống trên node — bất kể container bên trong bị restart bao nhiêu lần. Nó chỉ biến mất khi chính Pod đó bị xoá khỏi node. Đây chính là cơ chế mình dựa vào cho toàn bộ lời giải: ở Stage 1, người chơi upload payload vào `/app/cache`, một mount `emptyDir`. Ở Stage 3, mình buộc Pod tự sát bằng cách giết container. Và vì đó là container restart chứ không phải Pod bị xoá, `emptyDir` — cùng với payload bên trong — vẫn còn nguyên khi container mới khởi động lại ở Stage 4. Nếu `emptyDir` không có tính chất này, toàn bộ kill chain sẽ sụp đổ ngay tại bước 3.

[Gợi ý: thay `k8s-lifecycle-diagram.png` bằng sơ đồ vòng đời Pod thật]

Slide 10 — TOCTOU — What Is It?

Time-of-check to time-of-use.

Trong một seminar trước đây của bạn Ngô Bảo Duy Anh thì chúng ta đã được làm quen với khái niệm về TOCTOU tức là "Time-of-check to Time-of-use" nghĩa là bạn kiểm tra (validate) một giá trị, nhưng sau đó lại thao tác trên một lần đọc *khác* của giá trị đó — và trong khoảng thời gian giữa hai lần đó, dữ liệu gốc có thể đã thay đổi.

Slide 11 — TOCTOU — A Simple Example

Ví dụ kinh điển: kiểm tra quyền rồi mới mở file.

Để dễ hình dung hơn, có một ví dụ rất kinh điển trong bảo mật hệ điều hành. Một chương trình chạy quyền cao muốn ghi log vào một file, nên trước tiên nó kiểm tra xem file đó có đúng là sở hữu của user thường hay không — đây là bước "time-of-check". Nếu hợp lệ, nó mới mở file ra để ghi — đây là bước "time-of-use". Vấn đề là giữa hai bước đó có một khoảng hở rất nhỏ, và nếu kẻ tấn công kịp tráo file đó thành một symlink trỏ tới `/etc/passwd` ngay trong khoảng hở đó, thì chương trình vẫn tưởng mình đang ghi vào file log bình thường, nhưng thực chất lại đang ghi đè lên một file hệ thống quan trọng.

Slide 12 — TOCTOU — Last-Byte Synchronization

Kỹ thuật khai thác race condition qua HTTP.

Quay lại với web thì dạng TOCTOU kinh điển nhất là: kiểm tra một file, rồi đọc lại chính file đó để xử lý, thay vì dùng lại nội dung vừa kiểm tra. Kỹ thuật "last-byte-sync" là cách mình khiến khoảng hở đó có thể khai thác ổn định qua HTTP: giữ lại hai request đã chuẩn bị sẵn, mỗi request thiếu đúng byte cuối cùng, rồi thả cả hai byte cuối cùng ra gần như cùng lúc, để cả hai handler phía server bắt đầu xử lý trong cùng một khoảnh khắc.

[Gợi ý: thay `race-condition-diagram.png` bằng `timeline validate/promote` thật]

Slide 13 — DOM Clobbering — What Is It?

Phần tử HTML có tên đè lên biến global của JS.

DOM Clobbering là một chiêu chỉ dùng markup thuần túy, không cần một dòng JavaScript nào của kẻ tấn công cả. Nếu bạn đặt thuộc tính `id` cho một phần tử HTML, và chưa có biến JavaScript nào cùng tên được khai báo, trình duyệt sẽ tự động biến chính phần tử đó thành một biến global mang tên đó.

Slide 14 — DOM Clobbering — A Concrete Example

Vì sao bộ lọc HTML allow-list vẫn bị qua mặt.

Ví dụ trong challenge của mình, có một đoạn script hợp lệ tự tra cứu cấu hình qua biến `window.STEWARD_STATUS_CONFIG`. Nếu biến này chưa được khai báo, script sẽ dùng cấu hình mặc định. Nhưng nếu mình chèn một thẻ `<form id="STEWARD_STATUS_CONFIG">` vào bio của mình, trình duyệt sẽ biến chính thẻ form đó thành biến global cùng tên — và script kia sẽ đọc nhầm cấu hình của mình thay vì cấu hình mặc định. Đây chính là lý do một bộ lọc HTML dù đã cẩn thận loại bỏ script và các thuộc tính sự kiện, nhưng vẫn cho phép thẻ `<form>` và thuộc tính `id`, thì vẫn có thể bị lợi dụng — vì mình không hề chèn code, mình chỉ đang đánh lạc hướng nơi mà một đoạn script đáng tin cậy tự tra cứu cấu hình của chính nó.

[Gợi ý: thay `dom-clobbering-diagram.png` bằng sơ đồ clobbering thật]

Slide 15 — Insecure Deserialization — What Is It?

Serialize/deserialize, nguy hiểm khi bytes chọn được class.

Serialization biến một object thành chuỗi byte; deserialization biến chuỗi byte đó ngược lại thành object. Nguy hiểm nằm ở chỗ khi định dạng byte đó được phép chỉ định luôn *class* nào sẽ được khởi tạo, thay vì chỉ mang dữ liệu thuần túy.

Slide 16 — Insecure Deserialization — PyYAML Specifics

`yaml.Loader`, `!!python/object/apply`, so với `safe_load`.

`yaml.Loader` của PyYAML làm chính xác điều nguy hiểm đó — nó sẵn sàng dựng lên bất kỳ object Python nào từ một văn bản YAML. Tag `!!python/object/apply` còn đi xa hơn, gọi thẳng một hàm với tham số do kẻ tấn công chọn. Lựa chọn an toàn, `yaml.safe_load`, chỉ cho phép parse các kiểu dữ liệu thuần túy — không tag, không dựng object — và đây chính là hàm challenge của mình dùng để validate.

[Gợi ý: thay `deserialization-diagram.png` bằng sơ đồ tag-to-object thật]

4 Kiến trúc thử thách

Slide 17 — Section divider — Challenge Architecture

Slide chuyển mục, tự sinh.

Có nền tảng đó rồi, giờ mình sẽ đi vào cách mình xây dựng thử thách.

Slide 18 — Two-Tier Deployment

Web Pod + DB Pod, cụm Kind, vị trí flag.

Kiến trúc thì khá đơn giản: một Pod web chạy Flask và một Pod database chạy Postgres, cả hai triển khai trên một cụm Kind chạy local nên ai cũng dựng lại được y hệt trên máy mình. Điểm mình muốn nhấn mạnh là flag không hề nằm trong Pod web — nó chỉ tồn tại bên trong Postgres, nên người chơi bắt buộc phải khai thác thật sự ở tầng mạng, chứ không thể chỉ đọc trộm một file trên đĩa là xong.

[Gợi ý: thay `architecture-diagram.png` bằng sơ đồ triển khai thật]

Slide 19 — Defenses In Place

Hai mount `emptyDir`; vài guardrail chặn lỗi tắt.

Trên Pod web, hai nơi duy nhất ghi được là hai mount `emptyDir`: `/app/cache`, đúng chỗ payload của người chơi sẽ nằm ở Stage 1, và `/tmp`. Mình cũng có gắn thêm vài guardrail khác như `readOnlyRootFilesystem` hay `NetworkPolicy`, nhưng những cái đó chỉ để chặn các lỗi tắt không mong muốn thôi — không phải trọng tâm của lời giải, nên mình sẽ không đi sâu.

5 Chuỗi tấn công

Slide 20 — Section divider — The Kill Chain

Slide chuyển mục, tự sinh.

Giờ mình sẽ đi qua từng bước trong lời giải bốn giai đoạn.

Slide 21 — Overview

Sơ đồ kill chain: Stage, Race, Death, Return.

Nhìn tổng quan thì có bốn bước: Stage — cấy payload độc hại. Race — thắng một race condition kiểu TOCTOU để payload được "thăng cấp". Death — buộc Pod tự sát. Return — Pod quay lại và tự kích hoạt payload giúp mình. Giờ mình đi vào từng bước một.

[Gợi ý: thay `killchain-diagram.png` bằng sơ đồ 4 giai đoạn thật]

Slide 22 — Stage 1 — Payload Staging

Đăng ký, upload `profile.yml` độc hại dạng backup.

Người chơi đăng ký một tài khoản bình thường, rồi dùng tính năng "di trú hồ sơ cũ" để upload một file YAML. File này không đi thẳng vào file mà ứng dụng tin tưởng đâu — nó chỉ được lưu tạm thành `profile.staging.yml` trong cache `emptyDir`. Tự bản thân file này lúc này chưa làm được gì cả.

Slide 23 — Stage 2 — The Race

`/health` kiểm tra bằng `safe_load`, rồi đọc lại từ đĩa để promote.

`/health`, chính là readiness probe của Kubernetes, đang làm hai việc cùng lúc: kiểm tra file đang staged bằng `yaml.safe_load` — thứ sẽ từ chối thẳng payload độc hại của mình — và nếu kiểm tra đó thành công, nó "thăng cấp" file staged thành `profile.yml` chính thức bằng cách copy lại từ đĩa một lần nữa. Đây chính là khoảng hở TOCTOU mình vừa nói: bước promote đọc lại file từ đĩa thay vì dùng lại nội dung vừa kiểm tra. Nên người chơi sẽ staged một file placeholder vô hại trước, rồi dùng đúng kỹ thuật last-byte-sync để tráo vào payload độc hại ngay trong khoảng hở hẹp giữa lúc kiểm tra và lúc promote.

Slide 24 — Stage 3 — The Death

Clobber DOM trong bio, admin bot, `/maintenance/restart`, `os._exit`.

Bây giờ file độc hại đã nằm ở đúng vị trí `profile.yml` được tin tưởng rồi — nhưng file này chỉ được load đúng một lần lúc process khởi động thôi, nên người chơi cần ứng dụng phải khởi

động lại. Họ đặt một payload DOM Clobbering vào bio hồ sơ của mình, payload này ghi đè lên object cấu hình đồng bộ trạng thái mình vừa nói ở phần trước, rồi gửi một ticket hỗ trợ dẫn link tới hồ sơ đó. Một admin bot ghé thăm trang, script tự kiểm tra trạng thái của chính nó bị đánh lạc hướng bởi cấu hình đã bị clobber, và phiên đăng nhập đã xác thực của admin cuối cùng gửi request POST tới endpoint hạn chế `/api/v1/infra/maintenance/restart`. Handler đó chỉ đơn giản gọi `os._exit(1)` — Pod chết ngay tại chỗ.

Slide 25 — Stage 4 — The Return

Container restart, deserialize không an toàn, DatabaseExporter, exfil.

Kubernetes khởi động lại container, và đúng như mình nói ở phần Kubernetes 101, vì đây là container restart chứ không phải Pod bị xoá, nên volume `emptyDir` — cùng với `profile.yml` độc hại của mình — vẫn còn nguyên. Lúc khởi động, ứng dụng deserialize file đó bằng `yaml.Loader` không an toàn một cách vô điều kiện, đúng như mình giải thích ở phần deserialization. Payload khởi tạo một gadget class tên `DatabaseExporter`, chạy một câu truy vấn SQL tùy ý lên Postgres — một cách hợp lệ, đi đúng đường mà `NetworkPolicy` cho phép, vì request xuất phát từ chính Pod web — rồi POST kết quả, chính là flag, ra một webhook bên ngoài.

6 Lý do thiết kế

Slide 26 — Section divider — Design Rationale

Slide chuyển mục, tự sinh.

Một vài lưu ý về lý do mình xây dựng thử thách theo cách này.

Slide 27 — Anti-Slop Design

Endpoint giả, cổng chặn `safe_load`, buộc phải hiểu bản chất.

Mình có cài vài endpoint giả — metrics, logs, node status — để dụ mấy công cụ scan tự động, hay mấy bạn copy-paste từ LLM, đi lạc vào SSRF hay RCE không hề tồn tại. Nhưng quan trọng hơn cả là cổng chặn `safe_load` trên đường upload, khiến cho cách làm "hiển nhiên" nhất — upload thẳng một file YAML độc hại — sẽ thất bại ngay từ đầu. Người chơi buộc phải thực sự hiểu khoảng hở validate/promote thì mới đi tiếp được. Đó chính là mục tiêu thiết kế anti-slop của mình: thưởng cho việc hiểu hệ thống, chứ không phải đoán mò.

Slide 28 — Reliability

Race mang tính xác suất, thiết kế thử lại tới khi thành công.

Mình xin nói thẳng luôn: race ở Stage 2 mang tính xác suất, không phải tất định. Script giải mẫu của mình xử lý việc này bằng cách bắn race theo từng đợt và tự động thử lại, đồng thời chờ Pod hồi phục hoàn toàn giữa các lần thử. Mình thiết kế để một người chơi đã hiểu đúng lỗ hổng chắc chắn sẽ lấy được flag sau cùng, dù một lượt race đơn lẻ có thể không ăn.

7 Demo

Slide 29 — Section divider — Demo

Slide chuyển mục, tự sinh.

Giờ mình sẽ demo cho mọi người xem toàn bộ quy trình diễn ra trên thực tế.

Slide 30 — Live Demo

Gợi ý cho demo trực tiếp hoặc video ghi sẵn.

[Gợi ý: chạy demo trực tiếp tại đây, hoặc phát video ghi sẵn; thay demo-screenshot.png nếu chỉ dùng ảnh chụp tĩnh]

Bây giờ mình sẽ chạy script exploit mẫu lên cụm thật — đăng ký tài khoản, staged payload, thắng race, kích hoạt admin bot, và chờ Pod quay lại cùng với flag.

Slide 31 — Thank You

Slide kết thúc, thanks người thuyết trình.

Đó là toàn bộ chuỗi tấn công. Mình sẵn sàng trả lời câu hỏi — về thiết kế lỗ hổng, độ tin cậy của race, hay bất cứ điều gì trong mã nguồn.